Institución Universitaria Pascual Bravo

Base de Datos I

Tarea 4: Implementación en tres SGBD y datos semiestructurados

Red social estudiantil "Pascualina"

Grupo 9

Integrantes:

Andry Zharick Mariño Moreno

Alejandro Gallego Velásquez

2026

Introducción

En esta tarea tomamos el modelo lógico de la Tarea 2 y lo implementamos en tres SGBD diferentes: PostgreSQL, MySQL y SQL Server. Además, agregamos campos JSON para guardar datos semiestructurados y los relacionamos con casos de uso de Big Data e IoT. Al final dejamos el diccionario genérico actualizado, un diccionario físico y un script DDL por cada motor.

1. Justificación de los tres SGBD

Escogimos tres motores que son diferentes entre sí, para poder compararlos de verdad:

SGBD	Por qué lo elegimos	Programa con ventanas
PostgreSQL	Es gratis y ya lo habíamos usado en la Tarea 3. Cumple muy bien el estándar SQL y guarda JSON en binario (JSONB), lo que lo hace rápido de consultar.	pgAdmin 4
MySQL	Es gratis y de los más usados en aplicaciones web. Tiene un tipo JSON propio y sus reglas son un poco distintas a las de PostgreSQL.	MySQL Workbench
SQL Server	Es el motor de Microsoft, muy usado en empresas, y su versión Express es gratis. Maneja el JSON de otra forma (como texto validado), así que sirve para comparar.	SQL Server Management Studio (SSMS)

Con estos tres cubrimos un motor de código abierto muy completo, uno de código abierto muy popular y uno comercial. Como además cada uno guarda el JSON de manera diferente, la comparación es más útil para esta tarea. SSMS es la herramienta oficial de Microsoft para trabajar con SQL Server (Microsoft Ignite, 2025).

2. Diccionario de datos genérico actualizado

El diccionario genérico describe las tablas sin depender de ningún motor, con tipos generales: texto, entero, fecha, fecha y hora, y semiestructurado (JSON). Lo armamos a partir del diagrama ER de la Tarea 1 y del modelo lógico normalizado de la Tarea 2. Tiene 16 tablas, 87 campos, sus claves primarias, foráneas y únicas, y 22 relaciones. Está en Tarea4/diccionarios/diccionario-generico. Lo que se actualizó fue:

- El campo fecha de reaccion pasó a llamarse fecha_reaccion, para que se entienda mejor.
- Se agregó el campo lugar.datos_iot, de tipo semiestructurado (JSON).
- Se agregó el campo publicacion.metadatos, de tipo semiestructurado (JSON).

#	Tabla	Campos	Clave primaria	Llaves foráneas
1	estudiante	9	id_estudiante	0
2	perfil	8	id_perfil	1
3	habilidad	3	id_habilidad	0
4	interes	3	id_interes	0
5	estudiante_habilidad	5	id_estudiante_habilidad	2
6	estudiante_interes	4	id_estudiante_interes	2
7	grupo	7	id_grupo	1

#	Tabla	Campos	Clave primaria	Llaves foráneas
8	miembro_grupo	5	id_miembro	2
9	lugar	4	id_lugar	0
10	evento	9	id_evento	3
11	asistente_evento	5	id_asistente	2
12	publicacion	6	id_publicacion	1
13	comentario	5	id_comentario	2
14	reaccion	5	id_reaccion	2
15	seguir	4	id_seguir	2
16	mentoria	5	id_mentoria	2

3. Datos semiestructurados (JSON) para Big Data e IoT

3.1 Qué agregamos

Tabla.campo	Qué guarda	Ejemplo
lugar.datos_iot	Coordenadas del lugar y lecturas de sus sensores (caso IoT)	{"coordenadas": {"lat": 6.2442, "lng": -75.5812}, "sensores": {"aforo": 35, "temperatura": 24.5}}
publicacion.metadatos	Etiquetas, enlace y dispositivo de la publicación (caso Big Data)	{"etiquetas": ["sql", "bases_de_datos"], "enlace": "https://ejemplo.com/recurso", "dispositivo": "movil"}

3.2 Por qué JSON

Los datos de estos dos campos no siempre tienen la misma forma: una publicación puede tener etiquetas y enlace, otra solo la foto de un meme; un sensor manda el aforo y otro la temperatura. Si hiciéramos una columna por cada dato, quedarían muchas vacías y habría que cambiar la tabla cada vez que aparezca un dato nuevo. Con JSON cada registro guarda solo lo que tiene y la tabla no cambia.

3.3 Caso de uso de Big Data: publicacion.metadatos

Una red social recibe muchas publicaciones, de muchos tipos y todo el tiempo (volumen, variedad y velocidad, que son las características del Big Data). Las etiquetas, los enlaces y el dispositivo sirven para analizar qué temas se comparten más y para recomendar contenido. Además, se pueden agregar datos nuevos, como el idioma, sin modificar la tabla. Nosotros usamos una opción híbrida: seguimos en una base relacional, pero con una columna JSON. Esta idea se parece a los motores híbridos SQL y NoSQL del material de consulta (Herrera-Ramírez et al., 2021; Treviño-Villalobos et al., 2022) y a las comparaciones entre bases relacionales y NoSQL con datos semiestructurados (Fadhel y Jameel, 2022; Hasan et al., 2019).

3.4 Caso de uso de IoT: lugar.datos_iot

Imaginamos un campus con sensores en la biblioteca y en los salones que miden aforo y temperatura. Cada sensor manda pocos datos y distintos según el modelo, así que se guardan como JSON junto con las coordenadas del lugar. Así la aplicación puede mostrar qué lugar está menos lleno antes de programar un evento. Los valores son números pequeños, por lo que el dato ocupa poco espacio. Este campo solo guarda la última lectura; para guardar todo el historial haría falta otra solución (está en las propuestas de mejora).

3.5 Cómo lo guarda cada motor

Motor	Tipo usado	Cómo se valida	Cómo se consulta
PostgreSQL	JSONB	Revisa el formato al guardar	Operador ->> y se puede indexar (índice GIN)
MySQL	JSON	Revisa el formato al guardar	Operador ->> o JSON_EXTRACT
SQL Server	NVARCHAR(MAX)	Con una regla ISJSON que obliga a que sea JSON válido	Función JSON_VALUE

En PostgreSQL usamos JSONB y no JSON porque se guarda en binario, es más rápido de consultar y se puede indexar. En SQL Server usamos texto con validación porque el tipo json propio solo existe desde SQL Server 2025, y así el script funciona también en versiones anteriores (Microsoft Learn, s. f.).

4. Implementación en cada SGBD

4.1 Cómo pasamos del diccionario genérico al físico

- Partimos del diccionario genérico, con los tipos generales.
- Cambiamos cada tipo general por el tipo que usa el motor (tabla 4.2).
- Escribimos los CREATE TABLE en orden, primero las tablas padre y después las hijas, con sus llaves, reglas y relaciones (Mannino, 2007).
- Ajustamos lo propio de cada motor y lo documentamos en su diccionario físico, que tiene el tipo, el tamaño, las claves, las reglas y una nota por cada campo.

4.2 Tipos de dato en cada motor

Tipo genérico	PostgreSQL	MySQL	SQL Server
Entero autonumérico	SERIAL	INT AUTO_INCREMENT	INT IDENTITY(1,1)
Entero	INTEGER	INT	INT
Texto corto	VARCHAR(n)	VARCHAR(n)	NVARCHAR(n)
Texto largo	TEXT	TEXT	NVARCHAR(MAX)
Fecha	DATE	DATE	DATE
Fecha y hora	TIMESTAMP	DATETIME	DATETIME2
Semiestructurado	JSONB	JSON	NVARCHAR(MAX) + ISJSON
Fecha de hoy por defecto	CURRENT_DATE	(CURRENT_DATE)	(CAST(GETDATE() AS DATE))
Hora actual por defecto	CURRENT_TIMESTAMP	CURRENT_TIMESTAMP	SYSDATETIME()

4.3 Relaciones y reglas en cada motor

Tema	PostgreSQL	MySQL	SQL Server
Impedir borrar al padre	RESTRICT	RESTRICT	NO ACTION (no tiene RESTRICT)
Al actualizar una llave	CASCADE	CASCADE	NO ACTION (un IDENTITY no se actualiza)
Varios caminos de borrado en cascada	Sin problema	Sin problema	No se permite: 4 llaves pasaron a NO ACTION

Tema	PostgreSQL	MySQL	SQL Server
Reglas de valores (CHECK)	Siempre se aplican	Se aplican desde la versión 8.0.16	Siempre se aplican

Las cuatro llaves de SQL Server que cambiaron fueron comentario.id_estudiante, reaccion.id_estudiante, seguir.id_seguido y mentoria.id_aprendiz. Por eso, en SQL Server, antes de borrar un estudiante hay que borrar esos registros suyos.

4.4 PostgreSQL

Script: Script_PostgreSQL_Grupo9.sql. Diccionario físico: Diccionario_Fisico_PostgreSQL_Grupo9.xlsx.

- 1. Instalar PostgreSQL con pgAdmin 4.
- 2. Crear la base de datos pascualina.
- 3. Abrir el Query Tool y cargar el script.
- 4. Ejecutarlo completo: borra las tablas viejas, crea las 16 tablas con sus 22 relaciones, crea los índices del JSON y hace la prueba.
- 5. Verificar que aparezcan las 16 tablas y que las consultas del JSON devuelvan datos.
- 6. Revisar el resultado contra el diccionario físico.

Dificultades:

- Casi ninguna, porque ya lo habíamos usado en la Tarea 3. Lo nuevo fue JSONB y el índice GIN.

Ventajas:

- Es gratis y hace cumplir todas las reglas y relaciones.
- JSONB se puede indexar, así que las búsquedas dentro del JSON son rápidas.

Desventajas:

- La instalación y el manejo son más pesados al principio.

4.5 MySQL

Script: Script_MySQL_Grupo9.sql. Diccionario físico: Diccionario_Fisico_MySQL_Grupo9.xlsx.

- 1. Instalar MySQL Server 8 y MySQL Workbench.
- 2. Conectarse al servidor desde Workbench.
- 3. Abrir el script (File > Open SQL Script).
- 4. Ejecutarlo completo: crea la base de datos pascualina, las 16 tablas y hace la prueba del JSON.
- 5. Verificar en Schemas > pascualina > Tables (con Refresh) y con las consultas del JSON.
- 6. Revisar el resultado contra el diccionario físico.

Dificultades:

- Hubo que acordarse de ENGINE=InnoDB (para las llaves foráneas) y de utf8mb4 (para las tildes).
- El valor por defecto de la fecha lleva paréntesis y pide MySQL 8.0.13 o más nuevo; el campo JSON no acepta un valor por defecto común.
- No acepta la palabra CASCADE al borrar tablas, así que se borran en orden.

Ventajas:

- Es gratis, sencillo de usar y tiene tipo JSON propio.
- Se instala rápido y Workbench es fácil de manejar.

Desventajas:

- En versiones viejas ignora algunas reglas.
- Indexar el JSON es más complicado (hay que crear una columna generada).

4.6 SQL Server

Script: Script_SQLServer_Grupo9.sql. Diccionario físico: Diccionario_Fisico_SQLServer_Grupo9.xlsx.

- 1. Instalar SQL Server (Express o Developer) y SSMS.
- 2. Conectarse al servidor desde SSMS.
- 3. Abrir el script (File > Open > File).
- 4. Ejecutarlo completo con F5: crea la base de datos, las 16 tablas y hace la prueba del JSON.
- 5. Verificar en el Object Explorer (Databases > pascualina > Tables, con Refresh) y con las consultas del JSON.
- 6. Revisar el resultado contra el diccionario físico.

Dificultades:

- No deja dos caminos de borrado en cascada hacia la misma tabla, así que 4 relaciones pasaron a NO ACTION.
- No existe RESTRICT (se usa NO ACTION) y un IDENTITY no se puede actualizar.
- El JSON es texto con validación, y se necesita NVARCHAR y el separador GO.

Ventajas:

- Es muy completo y SSMS es una herramienta potente.
- Las funciones para leer JSON, como JSON_VALUE, son fáciles de usar.

Desventajas:

- La versión gratis tiene límites y SSMS solo funciona en Windows.
- El JSON es texto en las versiones anteriores a 2025.
- Pide más ajustes en las relaciones.

5. Conclusiones

Conclusión grupal

Con esta tarea vimos que un mismo modelo se puede implementar en varios motores, pero cada uno tiene sus propias reglas. El modelo lógico no cambió, pero el script sí: los tipos, el número automático, las fechas y, sobre todo, el borrado en cascada. También aprendimos que el JSON permite guardar datos que cambian de forma sin tocar la tabla y sin dejar de usar una base de datos relacional. PostgreSQL fue el más cómodo para nosotros, MySQL el más sencillo y SQL Server el que más ajustes pidió.

Conclusión de Andry

Con esta tarea entendí que un mismo modelo se puede escribir en varios motores, pero que cada uno tiene sus reglas. Lo más difícil fue entender por qué SQL Server no deja borrar en cascada por dos caminos, y que había que cambiar cuatro relaciones para que funcionara. También me llamó la atención que el JSON deja guardar datos que cambian de forma sin modificar la tabla. Me queda la duda de cuándo conviene usar JSON y cuándo una tabla aparte. Como mejora, me gustaría probar con muchos más datos para comparar la velocidad de los tres.

Conclusión de Alejandro

Lo que más me dejó fue ver que el modelo lógico no cambia, pero el script sí. Lo más complicado fue mantener las 16 tablas iguales en los tres motores y que cada diccionario físico coincidiera con su script. Me gustó el caso de IoT, porque el JSON sirve para guardar lecturas de sensores que no siempre tienen los mismos datos. Me queda la duda de cómo se guardaría el historial completo de un sensor, porque una columna JSON solo guarda la última lectura. Como mejora, haría pruebas automáticas de los scripts.

Propuestas de mejora

- Crear índices para buscar dentro del JSON también en MySQL y SQL Server (con columnas generadas o calculadas).
- Guardar el historial de lecturas de los sensores en una tabla aparte o en una base pensada para series de tiempo.
- Hacer un procedimiento en SQL Server que borre a un estudiante junto con sus comentarios, reacciones, seguimientos y mentorías.
- Probar los tres motores con muchos más datos para comparar su rendimiento.

7. Entregables y organización del repositorio

Todo está en el repositorio bd1-redsocial-grupo9, dentro de la carpeta Tarea4:

```
Tarea4/
  diccionarios/
    diccionario-generico/   (diccionario genérico actualizado)
    diccionario-sgbd-1/     (físico de PostgreSQL)
    diccionario-sgbd-2/     (físico de MySQL)
    diccionario-sgbd-3/     (físico de SQL Server)
  ddl/
    scripts-ddl-sgbd-1/     (script de PostgreSQL)
    scripts-ddl-sgbd-2/     (script de MySQL)
    scripts-ddl-sgbd-3/     (script de SQL Server)
  informe/                  (este informe en PDF)
  video/                    (enlace del video en YouTube)
```